

Informe de laboratorio 05

Tema: Árbol de Expansión Mínimo: Kruskal, Prim y Boruvka

Nota

Estudiante	Escuela	Asignatura
Capatinta Almanza Camilo B. A. ccapatinta@unsa.edu.pe	Escuela Profesional de Ingeniería de Sistemas	Lab. Análisis y Diseño de Algoritmos Semestre: IV Código: 1702231

Laboratorio	Tema	Duración
05	Árbol de Expansión Mínimo: Kruskal, Prim y Boruvka	02 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - B	01/10/2026	05/10/2026

1. Ejercicios/Problemas propuestos

1.1. Ejercicio 1

Implementa Kruskal con Union-Find (incluyendo compresión de caminos) para diseñar la red de cableado de menor costo que conecte un conjunto de edificios ingresados por el usuario.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Arista {
5     int u, v;
6     long long w;
7 };
8
9 struct UnionFind {
10     vector<int> padre;
11     vector<int> rango;
12     int componentes;
13
14     UnionFind(int n) : padre(n), rango(n, 0), componentes(n) {
15         iota(padre.begin(), padre.end(), 0);
16     }
17 }
```

```
16 }
17
18 int buscar(int x) {
19     if (padre[x] != x)
20         padre[x] = buscar(padre[x]);
21     return padre[x];
22 }
23
24 bool unir(int a, int b) {
25     a = buscar(a);
26     b = buscar(b);
27     if (a == b) return false;
28     if (rango[a] < rango[b]) swap(a, b);
29     padre[b] = a;
30     if (rango[a] == rango[b]) rango[a]++;
31     componentes--;
32     return true;
33 }
34 };
35
36 const char* EJEMPLO =
37     "6\n"
38     "Rectorado Biblioteca Laboratorio Cafeteria Gimnasio Auditorio\n"
39     "9\n"
40     "Rectorado Biblioteca 4\n"
41     "Rectorado Cafeteria 8\n"
42     "Biblioteca Laboratorio 8\n"
43     "Biblioteca Cafeteria 11\n"
44     "Laboratorio Gimnasio 7\n"
45     "Cafeteria Gimnasio 1\n"
46     "Cafeteria Auditorio 2\n"
47     "Gimnasio Auditorio 6\n"
48     "Laboratorio Auditorio 4\n";
49
50 int main() {
51
52     cout << "Usar datos de ejemplo? (s/n): ";
53     char op; cin >> op;
54     istreamstream ejemplo(EJEMPLO);
55     istream& in = (op == 's' || op == 'S') ? static_cast<istream&>(ejemplo) : cin;
56     if (op != 's' && op != 'S')
57         cout << "Ingrese n, luego los n nombres, luego m y luego m lineas 'A B costo':\n";
58
59     int n, m;
60     in >> n;
61     vector<string> nombre(n);
62     unordered_map<string, int> id;
63     for (int i = 0; i < n; i++) { in >> nombre[i]; id[nombre[i]] = i; }
64
65     in >> m;
66     vector<Arista> aristas;
67     for (int i = 0; i < m; i++) {
68         string a, b; long long c;
69         in >> a >> b >> c;
70         if (!id.count(a) || !id.count(b) || c < 0) {
71             cout << "Tendido invalido ignorado: " << a << " " << b << " " << c << "\n";
```

```
72     continue;
73 }
74 if (id[a] == id[b]) continue;
75 aristas.push_back({id[a], id[b], c});
76 }
77
78 sort(aristas.begin(), aristas.end(),
79      [](const Arista& x, const Arista& y) { return x.w < y.w; });
80
81 UnionFind uf(n);
82 vector<Arista> elegidas;
83 long long costoTotal = 0;
84
85 cout << "\n--- Recorrido de Kruskal (aristas por costo creciente) ---\n";
86 for (const Arista& e : aristas) {
87     if (uf.unir(e.u, e.v)) {
88         elegidas.push_back(e);
89         costoTotal += e.w;
90         cout << "ACEPTADA " << nombre[e.u] << " - " << nombre[e.v]
91              << " (costo " << e.w << ")\n";
92         if ((int)elegidas.size() == n - 1) break;
93     } else {
94         cout << "rechazada " << nombre[e.u] << " - " << nombre[e.v]
95              << " (costo " << e.w << " ) -> formaria un ciclo\n";
96     }
97 }
98
99 cout << "\n=== RED DE CABLEADO DE MENOR COSTO ===\n";
100 for (const Arista& e : elegidas)
101     cout << nombre[e.u] << " <-> " << nombre[e.v] << " costo " << e.w << "\n";
102 cout << "Costo total: " << costoTotal << "\n";
103
104 if (uf.componentes > 1)
105     cout << "AVISO: los tendidos dados no conectan todos los edificios; "
106          << "quedan " << uf.componentes << " grupos separados.\n";
107 return 0;
108 }
```

- Primero transforma nombres en indices numericos para ordenarlos de menor a mayor costo. Durante la ejecucion del metodo de ordenamiento, Union-Find verifica que los vertices esten en "grupos" separados, caso contrario formaria ciclos y estos los evita. Cuando llega a n-1 termina, entregando el costo total del MST que estuvo sumando durante todo el algoritmo.

```

camiloasd .../Unidad2/lab05/e-n main ! 21:08 g++ -std=c++17 e1.cpp -o e1 && ./e1
Usar datos de ejemplo? (s/n): s

--- Recorrido de Kruskal (aristas por costo creciente) ---
ACEPTADA Cafeteria - Gimnasio (costo 1)
ACEPTADA Cafeteria - Auditorio (costo 2)
ACEPTADA Rectorado - Biblioteca (costo 4)
ACEPTADA Laboratorio - Auditorio (costo 4)
rechazada Gimnasio - Auditorio (costo 6) -> formaria un ciclo
rechazada Laboratorio - Gimnasio (costo 7) -> formaria un ciclo
ACEPTADA Rectorado - Cafeteria (costo 8)

=== RED DE CABLEADO DE MENOR COSTO ===
Cafeteria <-> Gimnasio costo 1
Cafeteria <-> Auditorio costo 2
Rectorado <-> Biblioteca costo 4
Laboratorio <-> Auditorio costo 4
Rectorado <-> Cafeteria costo 8
Costo total: 19

```

Figura 1: Ejecución de e1.cpp con datos inicializados

```

camiloasd .../Unidad2/lab05/e-n main ! 21:08 g++ -std=c++17 e1.cpp -o e1 && ./e1
Usar datos de ejemplo? (s/n): n
Ingrese n, luego los n nombres, luego m y luego m líneas 'A B costo':
4
Biblioteca Rectorado Sistemas Cafeteria
5
Biblioteca Sistemas 10
Biblioteca Rectorado 6
Sistemas Rectorado 5
Sistemas Cafeteria 15
Rectorado Cafeteria 4

--- Recorrido de Kruskal (aristas por costo creciente) ---
ACEPTADA Rectorado - Cafeteria (costo 4)
ACEPTADA Sistemas - Rectorado (costo 5)
ACEPTADA Biblioteca - Rectorado (costo 6)

```

Figura 2: Ejecución de e1.cpp con datos ingresados por el usuario

1.2. Ejercicio 2

Implementa Prim con cola de prioridad y compara el árbol resultante (aristas y peso total) contra el obtenido con Kruskal sobre el mismo grafo.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Arista { int u, v; long long w; };
5
6 bool menor(const Arista& a, const Arista& b) {
7     if (a.w != b.w) return a.w < b.w;
8     if (a.u != b.u) return a.u < b.u;
9     return a.v < b.v;
10 }
11
12 struct UnionFind {
13     vector<int> padre, rango;

```

```
14 UnionFind(int n) : padre(n), rango(n, 0) { iota(padre.begin(), padre.end(), 0); }
15 int buscar(int x) { return padre[x] == x ? x : padre[x] = buscar(padre[x]); }
16 bool unir(int a, int b) {
17     a = buscar(a); b = buscar(b);
18     if (a == b) return false;
19     if (rango[a] < rango[b]) swap(a, b);
20     padre[b] = a;
21     if (rango[a] == rango[b]) rango[a]++;
22     return true;
23 }
24 };
25
26 vector<Arista> kruskal(int n, vector<Arista> aristas) {
27     sort(aristas.begin(), aristas.end(), menor);
28     UnionFind uf(n);
29     vector<Arista> res;
30     for (const Arista& e : aristas)
31         if (uf.unir(e.u, e.v)) res.push_back(e);
32     return res;
33 }
34
35 struct Candidato {
36     Arista e;
37     int destino;
38 };
39
40 struct MayorPrioridad {
41     bool operator()(const Candidato& a, const Candidato& b) const { return menor(b.e, a.e); }
42 };
43
44 vector<Arista> prim(int n, const vector<vector<Candidato>>& ady) {
45     vector<bool> visitado(n, false);
46     vector<Arista> res;
47     priority_queue<Candidato, vector<Candidato>, MayorPrioridad> pq;
48
49     for (int inicio = 0; inicio < n; inicio++) {
50         if (visitado[inicio]) continue;
51         visitado[inicio] = true;
52         for (const Candidato& c : ady[inicio]) pq.push(c);
53
54         while (!pq.empty()) {
55             Candidato c = pq.top(); pq.pop();
56             if (visitado[c.destino]) continue;
57             visitado[c.destino] = true;
58             res.push_back(c.e);
59             for (const Candidato& sig : ady[c.destino])
60                 if (!visitado[sig.destino]) pq.push(sig);
61         }
62     }
63     return res;
64 }
65
66 long long pesoTotal(const vector<Arista>& v) {
67     long long s = 0;
68     for (const Arista& e : v) s += e.w;
69     return s;
70 }
```

```

70 }
71
72 void imprimir(const string& titulo, const vector<Arista>& v) {
73     cout << titulo << "\n";
74     for (const Arista& e : v) cout << " " << e.u << " - " << e.v << " (peso " << e.w <<
    ↪ ") \n";
75     cout << " Peso total: " << pesoTotal(v) << "\n";
76 }
77
78 const char* EJEMPLO =
79     "7 11\n"
80     "0 1 7\n0 3 5\n1 2 8\n1 3 9\n1 4 7\n2 4 5\n3 4 15\n3 5 6\n4 5 8\n4 6 9\n5 6 11\n";
81
82 int main() {
83     cout << "Usar grafo de ejemplo? (s/n): ";
84     char op; cin >> op;
85     istringstream ejemplo(EJEMPLO);
86     istream& in = (op == 's' || op == 'S') ? static_cast<istream&>(ejemplo) : cin;
87     if (op != 's' && op != 'S') cout << "Ingrese 'n m' y luego m lineas 'u v w' (vertices
    ↪ 0..n-1):\n";
88
89     int n, m;
90     in >> n >> m;
91     vector<Arista> aristas;
92     vector<vector<Candidato>> ady(n);
93     for (int i = 0; i < m; i++) {
94         int u, v; long long w;
95         in >> u >> v >> w;
96         if (u == v || u < 0 || v < 0 || u >= n || v >= n) continue;
97         if (u > v) swap(u, v);
98         Arista e{u, v, w};
99         aristas.push_back(e);
100         ady[u].push_back({e, v});
101         ady[v].push_back({e, u});
102     }
103
104     vector<Arista> aK = kruskal(n, aristas);
105     vector<Arista> aP = prim(n, ady);
106
107     imprimir("=== KRUSKAL ===", aK);
108     imprimir("=== PRIM (cola de prioridad) ===", aP);
109
110     vector<Arista> ordK = aK, ordP = aP;
111     sort(ordK.begin(), ordK.end(), menor);
112     sort(ordP.begin(), ordP.end(), menor);
113     bool mismasAristas = ordK.size() == ordP.size() &&
114         equal(ordK.begin(), ordK.end(), ordP.begin(),
115             [](const Arista& a, const Arista& b) { return a.u == b.u && a.v == b.v && a.w
    ↪ == b.w; });
116
117     cout << "\n=== COMPARACION ===\n";
118     cout << "Peso Kruskal = " << pesoTotal(aK) << " | Peso Prim = " << pesoTotal(aP)
119         << (pesoTotal(aK) == pesoTotal(aP) ? " -> IGUALES\n" : " -> DIFERENTES
    ↪ (error)\n");
120     cout << "Aristas: " << (mismasAristas ? "ambos arboles tienen EXACTAMENTE las mismas
    ↪ aristas\n"

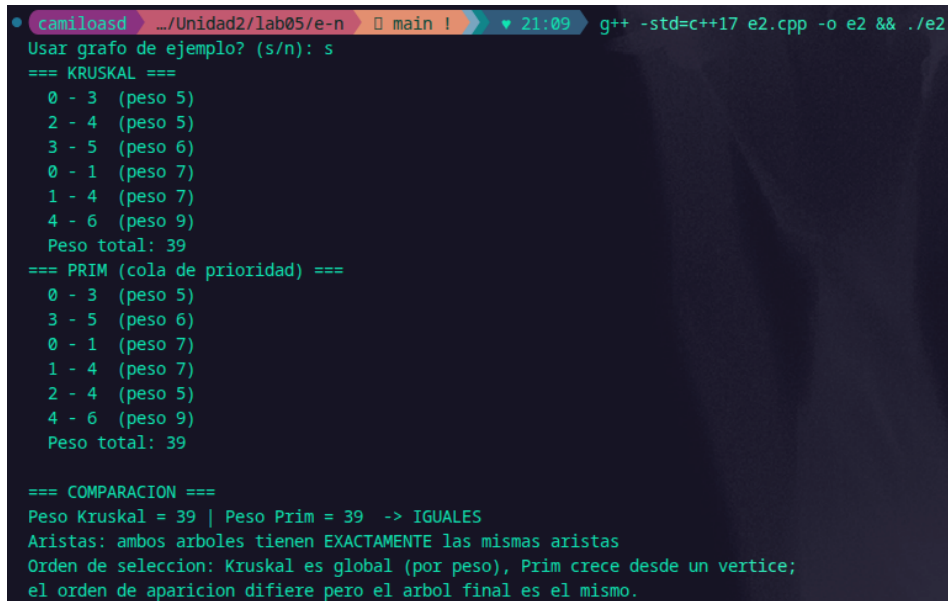
```

```

121                                     : "los conjuntos de aristas difieren\n");
122     cout << "Orden de seleccion: Kruskal es global (por peso), Prim crece desde un
    ↪     vertice;\n"
123         << "el orden de aparicion difiere pero el arbol final es el mismo.\n";
124     return 0;
125 }

```

- Compara los algoritmos Kruskal y Prim que resuelven el MST, Kruskal ya lo vimos, por su parte, Prim no encuentra sino que construye el MST mediante una cola de prioridad (el min-heap), teóricamente esto lo hace uniendo la raíz con un nuevo vértice no visitado.



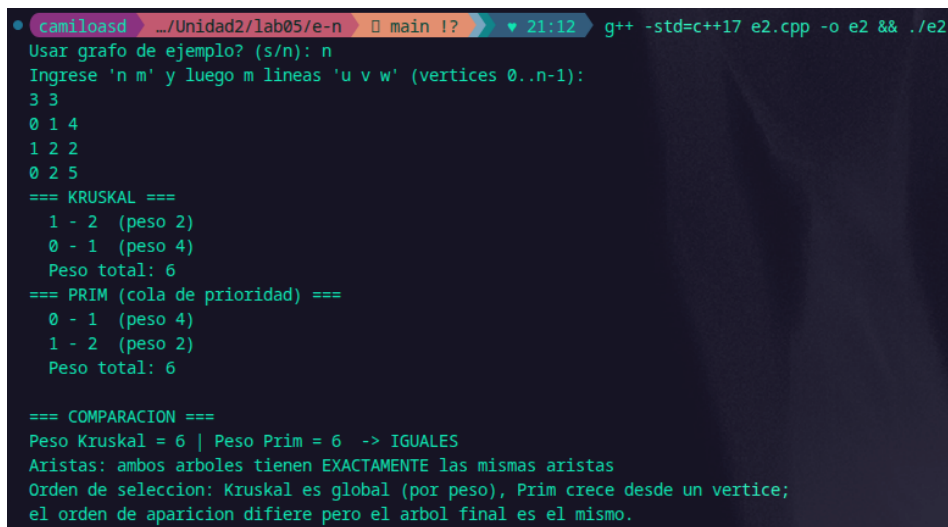
```

camiloasd ~/Unidad2/lab05/e-n [main !] 21:09 g++ -std=c++17 e2.cpp -o e2 && ./e2
Usar grafo de ejemplo? (s/n): s
=== KRUSKAL ===
0 - 3 (peso 5)
2 - 4 (peso 5)
3 - 5 (peso 6)
0 - 1 (peso 7)
1 - 4 (peso 7)
4 - 6 (peso 9)
Peso total: 39
=== PRIM (cola de prioridad) ===
0 - 3 (peso 5)
3 - 5 (peso 6)
0 - 1 (peso 7)
1 - 4 (peso 7)
2 - 4 (peso 5)
4 - 6 (peso 9)
Peso total: 39

=== COMPARACION ===
Peso Kruskal = 39 | Peso Prim = 39 -> IGUALES
Aristas: ambos arboles tienen EXACTAMENTE las mismas aristas
Orden de seleccion: Kruskal es global (por peso), Prim crece desde un vertice;
el orden de aparicion difiere pero el arbol final es el mismo.

```

Figura 3: Ejecución del e2.cpp con datos inicializados



```

camiloasd ~/Unidad2/lab05/e-n [main !?] 21:12 g++ -std=c++17 e2.cpp -o e2 && ./e2
Usar grafo de ejemplo? (s/n): n
Ingrese 'n m' y luego m lineas 'u v w' (vertices 0..n-1):
3 3
0 1 4
1 2 2
0 2 5
=== KRUSKAL ===
1 - 2 (peso 2)
0 - 1 (peso 4)
Peso total: 6
=== PRIM (cola de prioridad) ===
0 - 1 (peso 4)
1 - 2 (peso 2)
Peso total: 6

=== COMPARACION ===
Peso Kruskal = 6 | Peso Prim = 6 -> IGUALES
Aristas: ambos arboles tienen EXACTAMENTE las mismas aristas
Orden de seleccion: Kruskal es global (por peso), Prim crece desde un vertice;
el orden de aparicion difiere pero el arbol final es el mismo.

```

Figura 4: Ejecución del e2.cpp con datos ingresados por el usuario

1.3. Ejercicio 3

Implementa el algoritmo de Boruvka por rondas e imprime, en cada ronda, qué aristas se agregaron y cuántas componentes quedan, verificando que el resultado final coincide con el de Kruskal y Prim.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Arista { int u, v; long long w; };
5
6 bool menor(const Arista& a, const Arista& b) {
7     if (a.w != b.w) return a.w < b.w;
8     if (a.u != b.u) return a.u < b.u;
9     return a.v < b.v;
10 }
11
12 struct UnionFind {
13     vector<int> padre, rango;
14     int componentes;
15     UnionFind(int n) : padre(n), rango(n, 0), componentes(n) { iota(padre.begin(),
16     ↪ padre.end(), 0); }
17     int buscar(int x) { return padre[x] == x ? x : padre[x] = buscar(padre[x]); }
18     bool unir(int a, int b) {
19         a = buscar(a); b = buscar(b);
20         if (a == b) return false;
21         if (rango[a] < rango[b]) swap(a, b);
22         padre[b] = a;
23         if (rango[a] == rango[b]) rango[a]++;
24         componentes--;
25         return true;
26     };
27
28     long long pesoTotal(const vector<Arista>& v) {
29         long long s = 0;
30         for (auto& e : v) s += e.w;
31         return s;
32     }
33
34     vector<Arista> boruvka(int n, const vector<Arista>& E) {
35         UnionFind uf(n);
36         vector<Arista> res;
37         int ronda = 0;
38
39         cout << "Componentes iniciales: " << uf.componentes << "\n";
40         while (uf.componentes > 1) {
41             ronda++;
42
43             vector<int> mejor(n, -1);
44             for (int i = 0; i < (int)E.size(); i++) {
45                 int a = uf.buscar(E[i].u), b = uf.buscar(E[i].v);
46                 if (a == b) continue;
47                 if (mejor[a] == -1 || menor(E[i], E[mejor[a]])) mejor[a] = i;
48                 if (mejor[b] == -1 || menor(E[i], E[mejor[b]])) mejor[b] = i;
49             }
50 }
```

```
51     cout << "\n--- Ronda " << ronda << " ---\n";
52     int agregadas = 0;
53     for (int r = 0; r < n; r++) {
54         if (mejor[r] == -1) continue;
55         const Arista& e = E[mejor[r]];
56         if (uf.unir(e.u, e.v)) {
57             res.push_back(e);
58             agregadas++;
59             cout << " + arista " << e.u << " - " << e.v << " (peso " << e.w << ") \n";
60         }
61     }
62
63     cout << " Aristas agregadas: " << agregadas
64           << " | Componentes restantes: " << uf.componentes << "\n";
65
66     if (agregadas == 0) {
67         cout << " (No hay mas aristas salientes: el grafo no es conexo) \n";
68         break;
69     }
70 }
71 return res;
72 }
73
74 vector<Arista> kruskal(int n, vector<Arista> E) {
75     sort(E.begin(), E.end(), menor);
76     UnionFind uf(n);
77     vector<Arista> res;
78     for (auto& e : E) if (uf.unir(e.u, e.v)) res.push_back(e);
79     return res;
80 }
81
82 vector<Arista> prim(int n, const vector<Arista>& E) {
83
84     vector<vector<pair<int,int>>> ady(n);
85     for (int i = 0; i < (int)E.size(); i++) {
86         ady[E[i].u].push_back({i, E[i].v});
87         ady[E[i].v].push_back({i, E[i].u});
88     }
89
90     auto cmp = [&](const pair<int,int>& a, const pair<int,int>& b) { return
91         ↪ menor(E[b.first], E[a.first]); };
92     priority_queue<pair<int,int>, vector<pair<int,int>>, decltype(cmp)> pq(cmp);
93
94     vector<bool> vis(n, false);
95     vector<Arista> res;
96     for (int s = 0; s < n; s++) {
97         if (vis[s]) continue;
98         vis[s] = true;
99         for (auto& p : ady[s]) pq.push(p);
100        while (!pq.empty()) {
101            auto [idx, dest] = pq.top(); pq.pop();
102            if (vis[dest]) continue;
103            vis[dest] = true;
104            res.push_back(E[idx]);
105            for (auto& p : ady[dest]) if (!vis[p.second]) pq.push(p);
106        }
107    }
```

```

106     }
107     return res;
108 }
109
110 bool iguales(vector<Arista> a, vector<Arista> b) {
111     if (a.size() != b.size()) return false;
112     sort(a.begin(), a.end(), menor);
113     sort(b.begin(), b.end(), menor);
114     for (size_t i = 0; i < a.size(); i++)
115         if (a[i].u != b[i].u || a[i].v != b[i].v || a[i].w != b[i].w) return false;
116     return true;
117 }
118
119 const char* EJEMPLO =
120     "8 13\n"
121     "0 1 4\n0 2 3\n1 2 5\n1 3 6\n2 3 7\n2 4 8\n3 4 2\n3 5 9\n4 5 10\n4 6 11\n5 6 1\n5 7 12\n6
122     ↪ 7 13\n";
123
124 int main() {
125     cout << "Usar grafo de ejemplo? (s/n): ";
126     char op; cin >> op;
127     istringstream ejemplo(EJEMPLO);
128     istream& in = (op == 's' || op == 'S') ? static_cast<istream&>(ejemplo) : cin;
129     if (op != 's' && op != 'S') cout << "Ingrese 'n m' y luego m lineas 'u v w':\n";
130
131     int n, m;
132     in >> n >> m;
133     vector<Arista> E;
134     for (int i = 0; i < m; i++) {
135         int u, v; long long w;
136         in >> u >> v >> w;
137         if (u == v || u < 0 || v < 0 || u >= n || v >= n) continue;
138         if (u > v) swap(u, v);
139         E.push_back({u, v, w});
140     }
141
142     vector<Arista> aB = boruvka(n, E);
143     vector<Arista> aK = kruskal(n, E);
144     vector<Arista> aP = prim(n, E);
145
146     cout << "\n=== VERIFICACION FINAL ===\n";
147     cout << "Peso Boruvka = " << pesoTotal(aB) << "\n";
148     cout << "Peso Kruskal = " << pesoTotal(aK) << "\n";
149     cout << "Peso Prim = " << pesoTotal(aP) << "\n";
150     cout << "Boruvka == Kruskal (aristas): " << (iguales(aB, aK) ? "SI" : "NO") << "\n";
151     cout << "Boruvka == Prim (aristas): " << (iguales(aB, aP) ? "SI" : "NO") << "\n";
152     return 0;
153 }

```

- Se implementa el algoritmo de Boruvka, como es en la teoría, en cada ronda todas las componentes del grafo identifican simultáneamente su arista saliente menos costosa, su validación se aprueba mediante Union-Find. Esto hace que en cada paso el número de componentes reduzca significativamente.

```

camiloasd .../Unidad2/lab05/e-n main !? 21:14 g++ -std=c++17 e3.cpp -o e2 && ./e3
Usar grafo de ejemplo? (s/n): s
Componentes iniciales: 8

--- Ronda 1 ---
+ arista 0 - 2 (peso 3)
+ arista 0 - 1 (peso 4)
+ arista 3 - 4 (peso 2)
+ arista 5 - 6 (peso 1)
+ arista 5 - 7 (peso 12)
Aristas agregadas: 5 | Componentes restantes: 3

--- Ronda 2 ---
+ arista 1 - 3 (peso 6)
+ arista 3 - 5 (peso 9)
Aristas agregadas: 2 | Componentes restantes: 1

=== VERIFICACION FINAL ===
Peso Boruvka = 37
Peso Kruskal = 37
Peso Prim = 37
Boruvka == Kruskal (aristas): SI
Boruvka == Prim (aristas): SI

```

Figura 5: Ejecución de e3.cpp con datos inicializados

```

camiloasd .../Unidad2/lab05/e-n main !? 21:19 g++ -std=c++17 e3.cpp -o e3 && ./e3
Usar grafo de ejemplo? (s/n): n
Ingrese 'n m' y luego m líneas 'u v w':
6 6
0 1 1
2 3 2
4 5 3
1 2 10
3 4 11
0 5 20
Componentes iniciales: 6

--- Ronda 1 ---
+ arista 0 - 1 (peso 1)
+ arista 2 - 3 (peso 2)
+ arista 4 - 5 (peso 3)
Aristas agregadas: 3 | Componentes restantes: 3

--- Ronda 2 ---
+ arista 1 - 2 (peso 10)
+ arista 3 - 4 (peso 11)
Aristas agregadas: 2 | Componentes restantes: 1

=== VERIFICACION FINAL ===
Peso Boruvka = 27
Peso Kruskal = 27
Peso Prim = 27
Boruvka == Kruskal (aristas): SI
Boruvka == Prim (aristas): SI

```

Figura 6: Ejecución de e3.cpp con datos ingresados por el usuario

1.4. Ejercicio 4

Genera grafos aleatorios densos y dispersos de distintos tamaños y compara los tiempos de ejecución de Kruskal y Prim, determinando en qué casos conviene usar cada uno.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 using Reloj = chrono::steady_clock;
4
5 struct Arista { int u, v; int w; };
6
7 bool menor(const Arista& a, const Arista& b) {
8     if (a.w != b.w) return a.w < b.w;
9     if (a.u != b.u) return a.u < b.u;
10    return a.v < b.v;
11 }
12
13 vector<Arista> generarGrafo(int n, long long m, mt19937& rng) {
14     vector<Arista> E;
15     E.reserve(m);
16     vector<char> usada((size_t)n * n, 0);
17     uniform_int_distribution<int> peso(1, 1000000);
18
19     auto agregar = [&](int u, int v) {
20         if (u > v) swap(u, v);
21         usada[(size_t)u * n + v] = 1;
22         E.push_back({u, v, peso(rng)});
23     };
24
25     vector<int> perm(n);
26     iota(perm.begin(), perm.end(), 0);
27     shuffle(perm.begin(), perm.end(), rng);
28     for (int i = 1; i < n; i++) {
29         int j = uniform_int_distribution<int>(0, i - 1)(rng);
30         agregar(perm[i], perm[j]);
31     }
32
33     uniform_int_distribution<int> vert(0, n - 1);
34     while ((long long)E.size() < m) {
35         int u = vert(rng), v = vert(rng);
36         if (u == v) continue;
37         if (u > v) swap(u, v);
38         if (usada[(size_t)u * n + v]) continue;
39         agregar(u, v);
40     }
41     return E;
42 }
43
44 struct UnionFind {
45     vector<int> padre, rango;
46     UnionFind(int n) : padre(n), rango(n, 0) { iota(padre.begin(), padre.end(), 0); }
47     int buscar(int x) {
48         int r = x;
49         while (padre[r] != r) r = padre[r];
50         while (padre[x] != r) { int sig = padre[x]; padre[x] = r; x = sig; }
51         return r;
52     }
53     bool unir(int a, int b) {
54         a = buscar(a); b = buscar(b);
55         if (a == b) return false;
```

```
56         if (rango[a] < rango[b]) swap(a, b);
57         padre[b] = a;
58         if (rango[a] == rango[b]) rango[a]++;
59         return true;
60     }
61 };
62
63 long long kruskal(int n, vector<Arista> E) {
64     sort(E.begin(), E.end(), menor);
65     UnionFind uf(n);
66     long long total = 0;
67     int cnt = 0;
68     for (const Arista& e : E) {
69         if (uf.unir(e.u, e.v)) {
70             total += e.w;
71             if (++cnt == n - 1) break;
72         }
73     }
74     return total;
75 }
76
77 long long primCola(int n, const vector<Arista>& E) {
78
79     vector<vector<pair<int,int>>> ady(n);
80     for (const Arista& e : E) {
81         ady[e.u].push_back({e.v, e.w});
82         ady[e.v].push_back({e.u, e.w});
83     }
84     using Par = pair<int,int>;
85     priority_queue<Par, vector<Par>, greater<Par>> pq;
86     vector<char> vis(n, 0);
87     long long total = 0;
88     int cnt = 0;
89
90     pq.push({0, 0});
91     while (!pq.empty() && cnt < n) {
92         auto [w, u] = pq.top(); pq.pop();
93         if (vis[u]) continue;
94         vis[u] = 1; total += w; cnt++;
95         for (auto [v, pw] : ady[u])
96             if (!vis[v]) pq.push({pw, v});
97     }
98     return total;
99 }
100
101 long long primMatriz(int n, const vector<Arista>& E) {
102     const int INF = INT_MAX;
103     vector<int> mat((size_t)n * n, INF);
104     for (const Arista& e : E) {
105         mat[(size_t)e.u * n + e.v] = e.w;
106         mat[(size_t)e.v * n + e.u] = e.w;
107     }
108     vector<int> dist(n, INF);
109     vector<char> vis(n, 0);
110     long long total = 0;
111     dist[0] = 0;
```

```

112     for (int it = 0; it < n; it++) {
113         int u = -1;
114         for (int v = 0; v < n; v++)
115             if (!vis[v] && (u == -1 || dist[v] < dist[u])) u = v;
116         vis[u] = 1;
117         total += dist[u];
118         const int* fila = &mat[(size_t)u * n];
119         for (int v = 0; v < n; v++)
120             if (!vis[v] && fila[v] < dist[v]) dist[v] = fila[v];
121     }
122     return total;
123 }
124
125 template <class F>
126 double medir(F f, int reps, long long& resultado) {
127     double acum = 0;
128     for (int i = 0; i < reps; i++) {
129         auto t0 = Reloj::now();
130         resultado = f();
131         auto t1 = Reloj::now();
132         acum += chrono::duration<double, milli>(t1 - t0).count();
133     }
134     return acum / reps;
135 }
136
137 int main() {
138     mt19937 rng(12345);
139     const vector<int> tamanos = {100, 500, 1000, 2000};
140     const int REPS = 3;
141
142     cout << fixed << setprecision(3);
143     cout << left << setw(9) << "Tipo" << right << setw(7) << "n" << setw(10) << "m"
144         << setw(13) << "Kruskal(ms)" << setw(13) << "PrimCola(ms)" << setw(14) <<
145         << "PrimMatriz(ms)"
146         << " Mas rapido Pesos iguales\n";
147     cout << string(88, '-') << "\n";
148
149     vector<string> resumen;
150     for (string tipo : {"Disperso", "Denso"}) {
151         for (int n : tamanos) {
152             long long maxAristas = (long long)n * (n - 1) / 2;
153             long long m = (tipo == "Disperso") ? min(maxAristas, 3LL * n)
154                 : (long long)(0.7 * maxAristas);
155             vector<Arista> E = generarGrafo(n, m, rng);
156
157             long long tk, tp, tm;
158             double msK = medir([&] { return kruskal(n, E); }, REPS, tk);
159             double msP = medir([&] { return primCola(n, E); }, REPS, tp);
160             double msM = medir([&] { return primMatriz(n, E); }, REPS, tm);
161
162             string ganador = "Kruskal";
163             double mejor = msK;
164             if (msP < mejor) { mejor = msP; ganador = "PrimCola"; }
165             if (msM < mejor) { mejor = msM; ganador = "PrimMatriz"; }
166             bool ok = (tk == tp && tp == tm);

```

```

167         cout << left << setw(9) << tipo << right << setw(7) << n << setw(10) << E.size()
168         << setw(13) << msK << setw(13) << msP << setw(14) << msM
169         << " " << left << setw(12) << ganador << (ok ? "SI" : "NO (error)") <<
            << "\n";
170     }
171     cout << string(88, '-') << "\n";
172 }
173
174 cout << "\nCONCLUSIONES TEORICAS (contrastar con la tabla):\n"
175 << " - Grafo disperso ( $E \sim V$ ): Kruskal  $O(E \log E)$  y Prim-cola  $O(E \log V)$  son
176     << "     equivalentes;\n"
177 << "   Prim-matriz pierde por pagar  $O(V^2)$  y memoria  $V^2$ .\n"
178 << " - Grafo denso ( $E \sim V^2$ ):   Prim-matriz  $O(V^2)$  supera a los que usan log,
179     << "     pues\n"
180 << "   Kruskal ordena  $\sim V^2$  aristas ( $V^2 \log V$ ) y Prim-cola maneja un heap
        << "     enorme.\n";
return 0;
}

```

- Se comparan 3 algoritmos, Kruskal, Prim (con cola de prioridad) y Prim clasico (con matriz de adyacencia). Se genera grafos randoms de diferentes densidades, en grafos muy dispersos, es decir de baja densidad, Kruskal y Prim son mas eficientes por contraste: Prim clasico hace operaciones innecesarias basadas en el cuadrado del numero de vertices. Sin embargo, en grafos muy densos, Kruskal y Prim deben mantener un arbol muy grande, es costoso, Prim clasico no lidia con estos problemas.

camiloasd .../Unidad2/lab05/e-n 0 main !? 21:18 g++ -std=c++17 e4.cpp -o e3 && ./e4

Tipo	n	m	Kruskal(ms)	PrimCola(ms)	PrimMatriz(ms)	Mas rapido	Pesos iguales
Disperso	100	300	0.042	0.157	0.303	Kruskal	SI
Disperso	500	1500	0.278	0.787	3.738	Kruskal	SI
Disperso	1000	3000	0.345	1.246	15.356	Kruskal	SI
Disperso	2000	6000	0.807	2.634	63.597	Kruskal	SI
Denso	100	3465	0.293	0.558	0.173	PrimMatriz	SI
Denso	500	87325	9.907	12.120	4.101	PrimMatriz	SI
Denso	1000	349650	43.195	48.017	17.705	PrimMatriz	SI
Denso	2000	1399300	200.215	212.894	96.377	PrimMatriz	SI

CONCLUSIONES TEORICAS (contrastar con la tabla):

- Grafo disperso ($E \sim V$): Kruskal $O(E \log E)$ y Prim-cola $O(E \log V)$ son equivalentes; Prim-matriz pierde por pagar $O(V^2)$ y memoria V^2 .
- Grafo denso ($E \sim V^2$): Prim-matriz $O(V^2)$ supera a los que usan log, pues Kruskal ordena $\sim V^2$ aristas ($V^2 \log V$) y Prim-cola maneja un heap enorme.

Figura 7: Ejecución de e4.cpp

1.5. Ejercicio 5

Modela una red de distribución eléctrica (o de agua) entre varias localidades como un grafo ponderado por costo de tendido, y calcula el costo mínimo de conectarlas todas usando Kruskal o Prim.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3

```

```
4 struct Tramo {
5     int u, v;
6     double km;
7     double costo;
8 };
9
10 struct UnionFind {
11     vector<int> padre, rango;
12     int componentes;
13     UnionFind(int n) : padre(n), rango(n, 0), componentes(n) { iota(padre.begin(),
14     ↪ padre.end(), 0); }
15     int buscar(int x) { return padre[x] == x ? x : padre[x] = buscar(padre[x]); }
16     bool unir(int a, int b) {
17         a = buscar(a); b = buscar(b);
18         if (a == b) return false;
19         if (rango[a] < rango[b]) swap(a, b);
20         padre[b] = a;
21         if (rango[a] == rango[b]) rango[a]++;
22         componentes--;
23         return true;
24     }
25 };
26
27 const char* EJEMPLO =
28     "8\n"
29     "Yura Cayma Uchumayo Characato Sabandia Socabaya Chiguata Polobaya\n"
30     "14\n"
31     "Yura Cayma 12 60\n"
32     "Yura Uchumayo 18 75\n"
33     "Cayma Uchumayo 15 50\n"
34     "Cayma Socabaya 10 45\n"
35     "Uchumayo Socabaya 9 55\n"
36     "Socabaya Characato 8 40\n"
37     "Socabaya Sabandia 7 40\n"
38     "Characato Sabandia 5 35\n"
39     "Characato Chiguata 14 70\n"
40     "Sabandia Chiguata 17 80\n"
41     "Sabandia Polobaya 20 85\n"
42     "Chiguata Polobaya 16 90\n"
43     "Uchumayo Polobaya 30 95\n"
44     "Cayma Characato 13 50\n";
45
46 int main() {
47     cout << "Usar red de ejemplo? (s/n): ";
48     char op; cin >> op;
49     istringstream ejemplo(EJEMPLO);
50     istream& in = (op == 's' || op == 'S') ? static_cast<istream&>(ejemplo) : cin;
51     if (op != 's' && op != 'S')
52         cout << "Ingrese n, los nombres, m y luego m líneas 'A B km costo_por_km':\n";
53
54     int n, m;
55     in >> n;
56     vector<string> nombre(n);
57     unordered_map<string, int> id;
58     for (int i = 0; i < n; i++) { in >> nombre[i]; id[nombre[i]] = i; }
```

```

59     in >> m;
60     vector<Tramo> tramos;
61     double costoTodo = 0;
62     for (int i = 0; i < m; i++) {
63         string a, b; double km, cpk;
64         in >> a >> b >> km >> cpk;
65         if (!id.count(a) || !id.count(b) || km <= 0 || cpk < 0 || a == b) {
66             cout << "Tramo invalido ignorado: " << a << " " << b << "\n";
67             continue;
68         }
69         double costo = km * cpk;
70         tramos.push_back({id[a], id[b], km, costo});
71         costoTodo += costo;
72     }
73
74     sort(tramos.begin(), tramos.end(), [](const Tramo& x, const Tramo& y) {
75         if (x.costo != y.costo) return x.costo < y.costo;
76         if (min(x.u, x.v) != min(y.u, y.v)) return min(x.u, x.v) < min(y.u, y.v);
77         return max(x.u, x.v) < max(y.u, y.v);
78     });
79
80     UnionFind uf(n);
81     vector<Tramo> red;
82     double costoMin = 0, kmTotal = 0;
83     for (const Tramo& t : tramos) {
84         if (uf.unir(t.u, t.v)) {
85             red.push_back(t);
86             costoMin += t.costo;
87             kmTotal += t.km;
88             if ((int)red.size() == n - 1) break;
89         }
90     }
91
92     cout << fixed << setprecision(1);
93     cout << "\n=== RED DE DISTRIBUCION DE COSTO MINIMO ===\n";
94     for (const Tramo& t : red)
95         cout << setw(11) << nombre[t.u] << " <-> " << left << setw(11) << nombre[t.v] <<
96             << setw(6) << t.km << " km    costo " << setw(8) << t.costo << " (miles de
97             << S/)\n";
98
99     cout << "\nTramos contruidos : " << red.size() << " de " << tramos.size() << "
100         << posibles\n";
101     cout << "Longitud total      : " << kmTotal << " km\n";
102     cout << "COSTO MINIMO TOTAL : " << costoMin << " (miles de S/)\n";
103
104     if (uf.componentes > 1) {
105         cout << "\nAVISO: la red NO conecta todas las localidades; quedan " << uf.componentes
106             << " zonas aisladas. Se necesitan tramos adicionales.\n";
107     } else {
108         cout << "Costo de construir TODOS los tramos: " << costoTodo << " (miles de S/)\n";
109         cout << "Ahorro frente a construirlos todos : " << costoTodo - costoMin << " ("
110             << 100.0 * (costoTodo - costoMin) / costoTodo << "%)\n";
111     }
112     return 0;
113 }

```

- Se aplica Kruskal para resolver un problema de redes de distribución donde el costo por arista tiene un costo adicional de dinero, es por ello que se busca encontrar el MST. Primero calcula el costo financiero de cada tramo, luego con Union-Find se ordena de mayor a menor estos resultados y se aplica propiamente el algoritmo de Kruskal. Cuando se ejecuta simplemente calcula métricas del negocio, es una aplicación muy simple porque no toma en cuenta rutas alternativas en caso fallen algunas.

```
camiloasd .../Unidad2/lab05/e-n main l? 21:18 g++ -std=c++17 e5.cpp -o e3 && ./e3
Usar red de ejemplo? (s/n): s

=== RED DE DISTRIBUCION DE COSTO MINIMO ===
Characato <-> Sabandia      5.0 km costo 175.0 (miles de S/)
Socabaya <-> Sabandia      7.0 km costo 280.0 (miles de S/)
Cayma <-> Socabaya        10.0 km costo 450.0 (miles de S/)
Uchumayo <-> Socabaya      9.0 km costo 495.0 (miles de S/)
Yura <-> Cayma            12.0 km costo 720.0 (miles de S/)
Characato <-> Chiguata     14.0 km costo 980.0 (miles de S/)
Chiguata <-> Polobaya      16.0 km costo 1440.0 (miles de S/)

Tramos construidos : 7 de 14 posibles
Longitud total : 73.0 km
COSTO MINIMO TOTAL : 4540.0 (miles de S/)
Costo de construir TODOS los tramos: 13520.0 (miles de S/)
Ahorro frente a construirlos todos : 8980.0 (66.4%)
```

Figura 8: Ejecución de e5.cpp con datos inicializados

```
camiloasd .../Unidad2/lab05/e-n main l? 21:22 g++ -std=c++17 e5.cpp -o e5 && ./e5
Usar red de ejemplo? (s/n): n
Ingrese n, los nombres, m y luego m líneas 'A B km costo_por_km':
4
Cayma Yanahuara Socabaya Paucarpata
5
Cayma Yanahuara 3 20
Cayma Socabaya 10 45
Yanahuara Socabaya 8 40
Socabaya Paucarpata 6 30
Yanahuara Paucarpata 12 50

=== RED DE DISTRIBUCION DE COSTO MINIMO ===
Cayma <-> Yanahuara      3.0 km costo 60.0 (miles de S/)
Socabaya <-> Paucarpata  6.0 km costo 180.0 (miles de S/)
Yanahuara <-> Socabaya   8.0 km costo 320.0 (miles de S/)

Tramos construidos : 3 de 5 posibles
Longitud total : 17.0 km
COSTO MINIMO TOTAL : 560.0 (miles de S/)
Costo de construir TODOS los tramos: 1610.0 (miles de S/)
Ahorro frente a construirlos todos : 1050.0 (65.2%)
```

Figura 9: Ejecución de e5.cpp con datos ingresados por el usuario

2. Cuestionario

- ¿Por qué Kruskal necesita una estructura Union-Find, y qué problema resuelve la compresión de caminos dentro de esa estructura?

Kruskal agrega las aristas de menor a mayor peso, pero antes de agregar una debe saber si sus dos extremos ya están conectados, porque si lo están, la arista cerraría un ciclo. Revisar

eso recorriendo el grafo cada vez sería muy lento. Union-Find soluciona esto porque guarda los vértices en grupos y responde rápido a dos preguntas: “¿estos dos vértices están en el mismo grupo?” y “¿junta estos dos grupos?”.

La compresión de caminos resuelve un problema de velocidad. Para saber a qué grupo pertenece un vértice hay que subir por sus padres hasta llegar a la raíz, y esa cadena puede ser larga. Con la compresión, todos los vértices por los que se pasó quedan apuntando directamente a la raíz, así que las siguientes búsquedas son casi inmediatas.

2. ¿En qué se diferencia la estrategia de Prim de la de Kruskal, si ambos son algoritmos golosos?

Los dos son golosos porque en cada paso eligen la arista más barata que no forma un ciclo. La diferencia está en cómo van construyendo el árbol.

Kruskal revisa todas las aristas del grafo ordenadas por peso, y el resultado se va armando en varios pedazos que se unen poco a poco hasta formar un solo árbol. Prim, en cambio, hace crecer un único árbol desde un vértice inicial, y en cada paso elige la arista más barata que conecta ese árbol con un vértice que todavía no está dentro. Por eso Kruskal usa una lista de aristas ordenada junto con Union-Find, mientras que Prim usa una cola de prioridad.

3. ¿Puede un grafo tener más de un árbol de expansión mínimo? ¿Bajo qué condición ocurre esto?

Sí, puede ocurrir cuando hay aristas con el mismo peso, porque entonces existen varias formas de elegir y todas dan el mismo costo total. Si todos los pesos del grafo son distintos, el árbol de expansión mínimo es único.

Por ejemplo, en un triángulo donde las tres aristas pesan 1, se puede dejar fuera cualquiera de ellas, y así hay tres árboles posibles. Los tres pesan 2. Lo que nunca cambia es el peso total, que siempre es el mínimo.

4. ¿Por qué se dice que Boruvka es un algoritmo naturalmente paralelizable?

En cada ronda de Boruvka, cada componente elige su arista más barata sin necesitar saber qué están eligiendo las demás. Como esas decisiones son independientes entre sí, se pueden repartir entre varios procesadores o hilos y hacerlas al mismo tiempo.

Kruskal y Prim, en cambio, avanzan paso a paso, y cada decisión depende de lo que se decidió antes. Además, Boruvka necesita pocas rondas (como máximo $\log_2 n$), porque en cada una el número de componentes se reduce al menos a la mitad.

5. ¿Qué diferencia fundamental existe entre el problema del árbol de expansión mínimo y el problema del camino más corto visto en el Laboratorio 4?

El árbol de expansión mínimo busca conectar todos los vértices del grafo gastando lo menos posible en total, como cuando se quiere cablear todos los edificios de una universidad. El camino más corto, en cambio, busca la mejor ruta entre un origen y un destino específicos, como cuando se quiere ir de un punto a otro recorriendo la menor distancia.

Los resultados no son equivalentes. Si tenemos los vértices A, B y C con las aristas A-B de peso 1, B-C de peso 1 y A-C de peso 1.5, el árbol de expansión mínimo usa A-B y B-C, y para ir de A a C dentro del árbol hay que recorrer 2. Sin embargo, el camino más corto de A a C es la arista directa, que pesa 1.5.

3. Conclusiones

- Se implementaron tres algoritmos para encontrar el árbol de expansión mínimo: Kruskal (con Union-Find y compresión de caminos), Prim (con cola de prioridad) y Boruvka (por rondas). Los tres obtuvieron el mismo peso total sobre el mismo grafo, lo que confirma que el resultado mínimo es único aunque cada algoritmo llegue a él por un camino distinto.

- Union-Find es lo que hace eficiente a Kruskal, ya que permite detectar ciclos casi de inmediato. La compresión de caminos y la unión por rango evitan que los grupos se vuelvan cadenas largas y lentas de recorrer.
- Prim y Kruskal no siempre rinden igual. Kruskal depende de ordenar todas las aristas, por lo que se comporta bien en grafos con pocas conexiones, mientras que Prim suele convenir más cuando el grafo tiene muchas aristas, ya que solo trabaja con las que salen del árbol que va construyendo.
- Boruvka destaca porque sus decisiones en cada ronda son independientes, lo que lo hace muy adecuado para trabajar en paralelo. Además, mostrar las aristas agregadas y las componentes restantes en cada ronda ayudó a entender cómo el grafo se va uniendo poco a poco.
- Estos algoritmos tienen aplicaciones reales. Al modelar una red de distribución eléctrica entre localidades, se comprobó que construir solo los tramos del árbol de expansión mínimo permite conectar todas las localidades con un costo mucho menor que construir todos los tramos posibles.